

```

var http = require("http");

const server = http.createServer((request,response)=>{

btService.getInterface('/', 'org.freedesktop.DBus.ObjectManager',function(err,device_intf){
    device_intf.on('InterfacesAdded',function(devname){
        console.log(devname);
        response.statusCode = 200;
        response.setHeader('Content-Type', 'text/plain');
        response.write('New Device Path : '+devname);
        response.end();
    });
});
}).listen(8080);

console.log('Server started');

```

After running the application on the Node.js framework, open a browser and listen to port 8080 of the Web server. The browser will start listing the Bluetooth devices in the vicinity, as illustrated in Figure 5. 

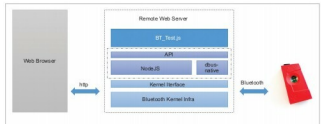


Figure 4: A representation of the Node.js HTTP server in the Linux Bluetooth ecosystem

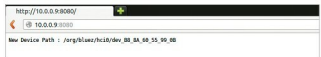


Figure 5: Node.js HTTP server response on a browser

By: Rajaram Regupathy

The author is an embedded software professional with hands-on experience in successfully delivering software/hardware products/solutions. He has published books and articles on various topics ranging from open source to FPGA which has got multiple citations.

Continued from Page 83...

IO-cache: This translator is used to cache the data that is read. This is useful if many applications read data multiple times and reads are more frequent than writes. The cache is maintained as an LRU list. When *io-cache* receives a write request, it will flush the file from the cache. It also periodically verifies the consistency of the cached files by checking the modification time on the file. The verification timeout is configurable.

Write-behind: This translator improves the latency of a write operation by relegating it to the background and returning to the application even as the write is in progress. Using the write-behind translator, successive write requests can be pipelined, which helps to reduce the network calls. Without the write-behind translator, once an application has issued a write call, it goes to the VFS and then to FUSE, which initiates a write call that is wound till it reaches the protocol client translator, which sends it to the server. Once the client receives a reply from the server, it returns it to the FUSE. With a write-behind translator, once the application issues a write call, it reaches VFS, then FUSE and then initiates the write call which is wound only till the write-behind translator. Write-behind adds the write buffer to its internal queue and returns with success. Write-behind initiates a fresh *writew()* call to its child translator, whose replies will be consumed by write-behind itself. Write-behind doesn't cache the write buffer.

Stat-prefetch: The purpose of this translator is to store the stat information. It helps to optimise operations like *ls*

-l, where a *readdir* is followed by a *stat* on the entry. This translator needs to maintain the following *fops*.

1. **Lookup:** Checks if the entry is present, and if so, then the stat information is returned from this translator rather than winding the call till the server. If the entry is not present, then we wind the lookup call and cache the stat information.
2. **Chmod/chown:** If a *chmod* or *chown* is performed, then the entry must be removed, since these calls change the *ctime* of directories.
3. **truncate/truncate/setxattr:** The entry must be removed, since these calls change the *mtime* of the directories.
4. **Rmdir:** Deletes the entry from the cache and then proceeds to remove the entry from the back-end. 

References

- [1] <https://gluster.readthedocs.io/en/latest/>
- [2] https://www.gluster.org/wp-content/uploads/2012/05/Gluster_File_System-3.3.0-Administration_Guide-en-US.pdf

By: Sakshi Bansal

The author has completed her bachelor's in computer science engineering from Amrita Vishwa Vidyapeetham. She has contributed to various open source projects such as Mozilla Thunderbird and Mediawiki. She blogs at <http://sakshiii.wordpress.com/>.